



AccessAI

Enterprise AI Accessibility Testing Agent

ENTERPRISE • v1.0.0

Complete Architecture, Implementation & Deployment Guide

FastAPI

Playwright

axe-core

Ollama

llama3

deepseek-r1

SQLite

Docker

100% Local • No Cloud APIs • Open Source Stack

May 2026

Table of Contents

1	Project Overview & Objectives
2	Architecture Overview
3	Tech Stack
4	Project Structure — 38 Files
5	Frontend Architecture (7 files)
6	Backend Architecture (14 files)
7	Infrastructure & DevOps (6 files)
8	AI Analysis Engine & Ollama Integration
9	Accessibility Scanner — axe-core + Playwright
10	API Reference
11	Sample AI JSON Output
12	Setup & Quick Start Guide
13	Docker Deployment
14	CI/CD Pipeline
15	Windows-Specific Notes
16	Security Implementation
17	Advanced Features Roadmap
18	Testing Guide
19	Troubleshooting
20	Resume-Ready Project Description

1. Project Overview & Objectives

AccessAI is a production-grade, enterprise-ready AI-powered web accessibility testing agent. It combines browser automation, the axe-core WCAG 2.1 engine, and locally-hosted Large Language Models (via Ollama) to scan websites, detect accessibility violations, and generate actionable remediation guidance — all running 100% locally with zero cloud dependencies.

Primary Objectives

- Accept any website URL from a professional dashboard UI
- Launch headless Chromium via Playwright for browser automation
- Inject and execute axe-core to detect WCAG 2.1 A/AA/AAA violations
- Classify violations by severity: Critical · High · Medium · Low
- Use local LLMs (llama3, deepseek-r1, qwen2.5, mistral) for AI analysis
- Generate human-readable explanations with business impact assessment
- Provide developer-ready code fix examples for each violation
- Calculate an overall accessibility score (0–100) with grade (A–F)
- Generate downloadable HTML and JSON reports
- Store scan history in SQLite for trend analysis
- Deploy with Docker Compose; integrate into CI/CD via GitHub Actions

100% Local | No Paid APIs | Open Source Only | WCAG 2.1 Compliant

2. Architecture Overview

AccessAI follows a clean three-tier architecture: a Vanilla JS frontend communicates with a FastAPI backend over REST/JSON. The backend orchestrates Playwright scanning, AI analysis via Ollama, and persistence to SQLite. All layers run on the local machine.

TIER	COMPONENT	TECHNOLOGY	ROLE
1 — Frontend	Dashboard UI	HTML5 · CSS3 · Vanilla JS	URL input, results dashboard, report download
2 — Backend	API Server	Python · FastAPI · Uvicorn	REST endpoints, scan orchestration, async handling
2 — Backend	Scanner	Playwright · axe-core 4.9	Browser automation, WCAG violation detection
2 — Backend	AI Engine	Ollama HTTP client	Prompt engineering, JSON parsing, retry logic
2 — Backend	Storage	SQLite ·aiosqlite	Scan history, results, screenshots paths
3 — AI Layer	LLM Runtime	Ollama localhost:11434	Runs llama3 / deepseek-r1 / qwen2.5 / mistral

Request Flow

- Step 1** User enters URL in dashboard → clicks Start AI Scan
- Step 2** Frontend POST /api/scan/start → backend creates DB record, returns scan_id
- Step 3** Background task: Playwright launches Chromium headless browser
- Step 4** Browser navigates to target URL, waits for DOM content loaded
- Step 5** axe-core injected via CDN script tag, runs WCAG audit
- Step 6** Violations array returned as JSON, classified by severity
- Step 7** Top violations sent to Ollama — AI generates structured analysis
- Step 8** Executive summary generated, score calculated, results saved to SQLite
- Step 9** Frontend polls /status every 1.5s, updates progress steps + bar
- Step 10** On completion, results displayed across 4 tabs: Violations / AI Analysis / Screenshot / Executive Summary

3. Tech Stack

<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'LAYER' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=8, greek=0, italic=0, link=[], rise=0, text='LAYER', textColor=Color(.545098,.580392,.619608,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>	<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'TECHNOLOGY' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=8, greek=0, italic=0, link=[], rise=0, text='TECHNOLOGY', textColor= Color(.545098,.580392,.619608,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>	<pre>Paragraph(' caseSensitiv e': 1 'encoding': 'utf8' 'text': 'VERSION' 'frags': [Para Frag(__tag__ ='b', bold=1, fontName='H elvetica-Bold ', fontSize=8, greek=0, italic=0, link=[], rise=0, text=' VERSION', te xtColor=Col or(.545098,.5 80392,.61960 8,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>	<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'PURPOSE' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=8, greek=0, italic=0, link=[], rise=0, text='PURPOSE', textColor=Color(.545098,.580392,.619608,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>
Frontend	HTML5 + CSS3 + Vanilla JS	—	Enterprise dashboard UI
Backend	Python + FastAPI	3.11 / 0.111	REST API, async, Pydantic
Server	Uvicorn	0.30	ASGI server with hot reload
Automation	Playwright (Chromium)	1.44	Headless browser control
A11y Engine	axe-core	4.9.1	WCAG 2.1 violation detection
AI Runtime	Ollama	latest	Local LLM server
AI Model 1	llama3	8B	Recommended, balanced speed/quality
AI Model 2	deepseek-r1	7B	Best analysis depth
AI Model 3	qwen2.5	7B	Strong multilingual support
AI Model 4	mistral	7B	Fast, efficient
AI Model 5	phi3:mini	3.8B	Fastest — good for quick scans
Database	SQLite + aiosqlite	—	Local async persistence
Validation	Pydantic v2	2.7	Request/response schemas
HTTP Client	httpx	0.27	Async Ollama API calls
Deploy	Docker + docker-compose	—	Containerised deployment
Proxy	Nginx	alpine	Frontend serving + API proxy

CI/CD	GitHub Actions	—	Lint, test, scan, build
-------	----------------	---	-------------------------

4. Project Structure — 38 Files

```
accessibility-ai-agent/
├── frontend/                                ← 7 files
│   ├── index.html
│   ├── styles/
│   │   ├── main.css
│   │   ├── dashboard.css
│   │   └── components.css
│   ├── scripts/
│   │   ├── api.js
│   │   ├── ui.js
│   │   ├── scanner.js
│   │   └── app.js
│   └── backend/                            ← 14 files
│       ├── api/routes/
│       │   ├── scan.py
│       │   ├── report.py
│       │   └── health.py
│       ├── services/
│       │   ├── scan_service.py
│       ├── scanner/
│       │   ├── axe_scanner.py
│       ├── ai_engine/
│       │   ├── ollama_service.py
│       ├── models/
│       │   ├── schemas.py
│       ├── utils/
│       │   ├── config.py
│       │   ├── database.py
│       │   ├── logger.py
│       │   └── validators.py
│       └── docker/                        ← 3 files
│           ├── docker-compose.yml
│           ├── Dockerfile.backend
│           └── nginx.conf
├── tests/
│   └── test_main.py
├── .github/workflows/ci.yml
├── main.py
├── requirements.txt
└── .env.example
```

5. Frontend Architecture (7 files)

FILE	DESCRIPTION
index.html	Main enterprise dashboard. URL input, progress steps, score ring animation, violations table, AI analysis cards, screenshot tab, executive summary, modal dialogs, toast notifications, dark/light theme.
styles/main.css	Design system: CSS variables for dark/light themes, layout, typography, navbar, sidebar, scrollbar styling. Defines all colour tokens (--critical, --high, --medium, --low, --accent).
styles/dashboard.css	All UI components: score ring SVG animation with grade colours, metric cards, severity badges, filter chips, tab system, violation table, AI analysis cards, modal overlay, toast notifications.
styles/components.css	Responsive breakpoints (mobile/tablet/desktop), utility classes, skeleton loader animation, pagination, empty states.
scripts/api.js	API client module: all fetch() calls to backend, polling helper (pollScan), report download with Blob URL, error handling.
scripts/ui.js	All rendering functions: score ring animation, metrics, violations table, AI analysis cards, executive summary, violation modal, scan history, screenshot display, toast helper, HTML escaping.
scripts/scanner.js	Client-side filter and search logic: severity filters, WCAG category filters, text search across violation fields.
scripts/app.js	Main controller: event binding, scan lifecycle management, tab switching, Ollama status polling, history loading, theme toggle.

Dashboard Features

- Score ring with animated SVG stroke and grade (A/B/C/D/F) coloured by threshold
- 5 metric cards: Critical / High / Medium / Low / Passed with colour-coded left borders
- 4-tab results panel: Violations · AI Analysis · Screenshot · Executive Summary
- Violations table with severity badges, WCAG tags, element count, detail modal
- Detail modal showing failing HTML snippet + AI explanation + code fix
- AI Analysis cards with Business Impact, Affected Users, Technical Explanation, Code Fix
- Executive Summary with Overview, Key Findings, Top 5 Recommendations, Compliance Status
- Severity filter chips and full-text search across violations
- Scan history sidebar with score and date for up to 10 previous scans
- Real-time progress bar with 6 labelled steps

6. Backend Architecture (14 files)

FILE	DESCRIPTION
<code>main.py</code>	FastAPI application entry point. Sets Windows ProactorEventLoop policy, registers CORS middleware, logging middleware, global exception handler, mounts static file serving for screenshots/reports.
<code>api/routes/scan.py</code>	5 REST endpoints: POST /start, GET /{id}/status, GET /{id}/results, GET /{id}/screenshot, GET /s/history. Uses BackgroundTasks for non-blocking scan execution.
<code>api/routes/report.py</code>	Report generation: HTML report with dark-themed styling, JSON export. Generates and serves files from data/reports/. Includes metric summary, violations table, AI remediation cards.
<code>api/routes/health.py</code>	GET /health (app status) and GET /health/ollama (pings Ollama, returns available models list).
<code>services/scan_service.py</code>	Scan orchestration: creates DB record, runs background pipeline, manages in-memory status dict for real-time polling, saves partial/final results to SQLite.
<code>scanner/axe_scanner.py</code>	Playwright scanner using sync_playwright in ThreadPoolExecutor (Windows fix). CDN axe-core injection, WCAG config per level, severity mapping from axe impact, score algorithm, deduplication, multi-page crawl.
<code>ai_engine/ollama_service.py</code>	Ollama HTTP client with httpx: 3 prompt templates (violation analysis, executive summary, developer summary), retry with exponential backoff, robust JSON parsing (handles markdown fences, embedded JSON), fallback analysis when offline.
<code>models/schemas.py</code>	All Pydantic v2 models: ScanRequest, ScanStatusResponse, ScanResultsResponse, Violation, AiAnalysis, ScanMetrics, ExecutiveSummary, HistoryResponse.
<code>utils/config.py</code>	Settings via pydantic-settings with .env file support. All timeouts, paths, model names, feature flags as typed fields.
<code>utils/database.py</code>	SQLAlchemy 2.0 async with aiosqlite. ScanRecord ORM model with JSON columns for violations/analyses. init_db() creates tables on startup.
<code>utils/logger.py</code>	Centralised logging: file + stdout handlers, DEBUG in dev, INFO in prod, suppresses noisy Playwright/httpx loggers.
<code>utils/validators.py</code>	URL validation: scheme whitelist, hostname check, length limit, null-byte stripping. SSRF protection (commented out for local dev).

7. Infrastructure & DevOps (6 files)

FILE	DESCRIPTION
<code>docker/docker-compose.yml</code>	3-service stack: accessai-backend (Python/Playwright), accessai-ollama (LLM server with persistent volume), accessai-nginx (frontend + reverse proxy). Shared bridge network.
<code>docker/Dockerfile.backend</code>	python:3.11-slim base. Installs Playwright system deps (Chromium, fonts, audio libs), Python packages, Playwright browsers. Creates data directories.
<code>docker/nginx.conf</code>	Serves frontend static files. Proxies /api/ and /health to backend:8000 with 300s timeout for long scans.
<code>.github/workflows/ci.yml</code>	4-job pipeline: lint (ruff) → unit tests (pytest) → accessibility regression scan on push to main → Docker image build.
<code>tests/test_main.py</code>	Pytest suite: route tests with mocked services, unit tests for scanner severity mapping, score calculation, URL validation, JSON parser robustness.
<code>.env.example</code>	All 16 environment variables documented with defaults: Ollama URL/model/timeout, browser headless flag, scan timeout, screenshot toggle, rate limits.

8. AI Analysis Engine & Ollama Integration

The AI engine uses structured prompt engineering to generate actionable, developer-ready accessibility remediation guidance. All LLM calls are made to the local Ollama server with retry logic, timeout handling, and robust JSON extraction.

Supported Models

<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'MODEL' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=8, greek=0, italic=0, link=[], rise=0, text='MODEL', textColor=Color(.545098,.580392,.619608,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>	<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'SPEED' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=8, greek=0, italic=0, link=[], rise=0, text='SPEED', textColor=Color(.545098,.580392,.619608,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>	<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'BEST FOR' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=8, greek=0, italic=0, link=[], rise=0, text='BEST FOR', textColor=Color(.545098,.580392,.619608,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>	
llama3	8B	★★★★	Recommended — balanced speed and quality
deepseek-r1	7B	★★★	Best analysis depth and reasoning
qwen2.5	7B	★★★★	Strong multilingual, good JSON output
mistral	7B	★★★★	Fast, reliable structured output
phi3:mini	3.8B	★★★★★	Fastest — good for rapid scans

Prompt Templates

Three prompt templates are used:

Violation Analysis: Analyzes a single violation — returns structured JSON with issue title, WCAG reference, business impact, affected users, technical explanation, recommended fix, code example, and priority.

Executive Summary: Takes aggregate scan data — returns JSON with overview, key findings, top 5 recommendations, compliance status, and score reasoning.

Developer Summary: Generates plain-text developer-focused remediation guide with priority ordering and effort estimates.

Retry & Error Handling

Each Ollama call uses exponential backoff (2^n seconds) with up to 3 retries. JSON is extracted robustly — handles clean JSON, markdown code fences (``json), and JSON embedded in surrounding text. Falls back to a pre-defined analysis template when Ollama is unreachable, so the app always returns useful results.

9. Accessibility Scanner — axe-core + Playwright

The scanner uses Playwright's sync API running inside a ThreadPoolExecutor to avoid Windows asyncio subprocess limitations. axe-core is injected from CDN and executed inside the live browser context.

Violation Types Detected

- Missing alt text on images (WCAG 1.1.1)
- Insufficient colour contrast (WCAG 1.4.3)
- Missing form labels (WCAG 1.3.1)
- Keyboard navigation failures (WCAG 2.1.1)
- ARIA attribute violations (WCAG 4.1.2)
- Missing document language (WCAG 3.1.1)
- Heading structure problems (WCAG 1.3.1)
- Link purpose unclear (WCAG 2.4.4)
- Missing page title (WCAG 2.4.2)
- Bypass blocks missing (WCAG 2.4.1)
- Focus order problems (WCAG 2.4.3)
- 70+ additional axe-core rules

Severity Classification

<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'AXE IMPACT' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helveti ca-Bold', fontSize=8, greek=0, italic=0, link=[], rise=0, text='AXE IMPACT', textColor=Color(.545098,.580392,.619608,1) , us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>	<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'ACCESSAI SEVERITY' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=8, greek=0, italic=0, link=[], rise=0, text='ACCESSAI SEVERITY', t extColor=Color(.545098,.5803 92,.619608,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>	<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'SCORE PENALTY' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=8, greek=0, italic=0, link=[], rise=0, text='SCORE PENALTY', textColor=Color(.545098,.580392,.619608,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>
critical	Critical	−10 per element
serious	High	−5 per element
moderate	Medium	−2 per element
minor	Low	−1 per element

Score Formula

Score = $100 \times (\text{passed_checks}) / (\text{passed_checks} + \text{weighted_violations})$ where weights are: Critical=10, High=5, Medium=2, Low=1. Score maps to grades: A (≥ 90) · B (≥ 75) · C (≥ 60) · D (≥ 40) · F (< 40).

10. API Reference

<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'METHOD' 'frags': [ParaFrag(__tag__='b' , bold=1, fontName='Helvetica-Bold', fontSize=8, greek=0, italic=0, link=[], rise=0, text='METHOD', textColor=Color(.545098,.580392,.619608,1), us_lines=[]) 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>	<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'ENDPOINT' 'frags': [ParaFrag(__tag__='b' , bold=1, fontName='Helvetica-Bold', fontSize=8, greek=0, italic=0, link=[], rise=0, text='ENDPOINT', textColor=Color(.545098,.580392,.619608,1), us_lines=[]) 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>	<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'DESCRIPTION' 'frags': [ParaFrag(__tag__='b' , bold=1, fontName='Helvetica-Bold', fontSize=8, greek=0, italic=0, link=[], rise=0, text='DESCRIPTION', textColor=Color(.545098,.580392,.619608,1), us_lines=[]) 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>
GET	/health	App health check
GET	/health/ollama	Ollama status + available models
POST	/api/scan/start	Start scan — returns scan_id immediately
GET	/api/scan/{id}/status	Poll progress: status, step, progress %
GET	/api/scan/{id}/results	Full results: violations, AI analyses, metrics
GET	/api/scan/{id}/screenshot	PNG screenshot of scanned page
GET	/api/scans/history	Last 20 scans with score and date
GET	/api/report/{id}?format=html	Download styled HTML report
GET	/api/report/{id}?format=json	Download raw JSON report
GET	/docs	Interactive Swagger UI documentation

POST /api/scan/start — Request Body

```
{
  "url":      "https://example.com",    // required
  "model":    "llama3",                 // llama3 | deepseek-r1 | qwen2.5 | mistral
  "depth":    1,                       // 1-5 pages
  "wcag_level": "AA"                   // A | AA | AAA
}
```

11. Sample AI Analysis JSON Output

```
{
  "issue_title": "Image missing alternative text",
  "severity": "critical",
  "wcag_reference": "WCAG 2.1 SC 1.1.1 – Non-text Content (Level A)",
  "business_impact": "Excludes approximately 2.2 billion users with visual impairments globally. Creates legal risk under ADA, AODA, and EN 301 549. Reputational damage from accessibility complaints.",
  "affected_users": "Blind and low-vision users relying on screen readers (JAWS, NVDA, VoiceOver). Images will be announced as 'unlabelled image' or skipped entirely.",
  "technical_explanation": "The <img> element at position #hero-image has no alt attribute. Screen readers cannot convey the image content. axe-core rule: image-alt (WCAG 1.1.1 Level A).",
  "recommended_fix": "1. Add descriptive alt text: alt='Company logo – returns to homepage'. 2. For decorative images use alt=''. 3. For complex images use aria-describedby pointing to a detailed description element.",
  "sample_code_fix": "<img src='logo.png' alt='AccessAI – returns to homepage'>",
  "priority": "P1"
}
```

Executive Summary JSON Output

```
{
  "overview": "The scan of example.com scored 85.7/100, indicating a strong accessibility foundation with one high-severity violation requiring attention.",
  "key_findings": "Primary issue: insufficient colour contrast on Sign In button (19 affected elements). 30 checks passed successfully.",
  "recommendations": [
    "Update Sign In button colours to meet 4.5:1 contrast ratio",
    "Audit all images for descriptive alt text",
    "Add ARIA landmarks to improve screen reader navigation",
    "Verify keyboard focus indicators are visible",
    "Test with screen reader software (NVDA/VoiceOver)"
  ],
  "compliance_status": "Partially meets WCAG 2.1 AA. Fix contrast issue to achieve full compliance.",
  "score_reasoning": "Score 85.7/100: 30 passed checks, 1 high violation (weight 5)."
}
```


12. Setup & Quick Start Guide

Prerequisites

```
Python 3.11+ • Git • Ollama installed (https://ollama.ai)
```

1. Clone & create venv

```
git clone <repo> && cd accessibility-ai-agent
python -m venv venv
venv\Scripts\activate (Windows)
```

2. Install dependencies

```
pip install -r requirements.txt
python -m playwright install chromium
```

3. Start Ollama

```
ollama serve
# In a second terminal:
ollama pull llama3
```

4. Configure

```
cp .env.example .env
# Defaults work for local development – no changes needed
```

5. Start backend

```
python main.py
# Backend runs at http://localhost:8000
```

6. Open frontend

```
open frontend/index.html
# Or: python -m http.server 3000 --directory frontend
```

7. Scan!

```
Enter URL → Select model → Click Start AI Scan
Results appear across 4 tabs when complete
```

13. Docker Deployment

The docker-compose stack runs three services on a shared bridge network: the FastAPI backend, Ollama LLM server (with persistent model storage volume), and Nginx serving the frontend and proxying API requests.

```
# Build and start all services
cd docker
docker-compose up --build

# Pull a model into the Ollama container (first time)
docker exec accessai-ollama ollama pull llama3

# Open the app
open http://localhost          # nginx serves frontend on port 80

# View logs
docker-compose logs -f backend
docker-compose logs -f ollama

# Stop
docker-compose down
```

Services

accessai-backend	FastAPI + Playwright — port 8000 (internal)
accessai-ollama	Ollama LLM server — port 11434, volume: ollama-data
accessai-nginx	Frontend + reverse proxy — port 80 (external)

14. CI/CD Pipeline — GitHub Actions

The pipeline runs on every push to main/develop and on all pull requests. Four jobs run in sequence: lint → test → accessibility regression → docker build.

```
# .github/workflows/ci.yml

Jobs:
  1. test      - ruff lint + pytest (all branches)
  2. a11y-scan - automated accessibility regression scan (main only)
                  fails build if critical violations > 0
  3. docker    - builds Docker image, tags with git SHA (main only)

# Add to GitHub repo settings:
Settings → Variables → SCAN_TARGET_URL = https://your-site.com
```

15. Windows-Specific Notes

IMPORTANT: Windows requires special configuration for Playwright + FastAPI

Windows uses SelectorEventLoop by default which cannot spawn subprocesses — required by Playwright's async API. The solution implemented in AccessAI is to run Playwright's sync_playwright inside a ThreadPoolExecutor, completely bypassing the event loop conflict.

The Fix (axe_scanner.py)

```
# sync_playwright runs in a thread – avoids asyncio subprocess conflict
def _run_playwright_sync(url, wcag_level, depth, screenshot_path):
    from playwright.sync_api import sync_playwright
    with sync_playwright() as p:
        browser = p.chromium.launch(headless=True)
        # ... scan logic

class AccessibilityScanner:
    async def scan(self, url, ...):
        loop = asyncio.get_event_loop()
        with ThreadPoolExecutor(max_workers=1) as pool:
            result = await loop.run_in_executor(
                pool, _run_playwright_sync, url, wcag_level, depth, screenshot_path
            )
```

Always start the server with `python main.py` rather than `uvicorn main:app` — this ensures the Windows event loop policy is set before uvicorn initialises.

16. Security Implementation

URL Validation	Scheme whitelist (http/https only), max length 2048, null-byte stripping, hostname validation, SSRF protection layer.
Input Sanitization	All URL inputs sanitized before reaching scanner. Pydantic v2 validates all API request bodies with strict types.
No External Calls	Zero cloud API calls — fully air-gapped. Ollama runs locally. No telemetry or data sharing.
Rate Limiting	RATE_LIMIT_SCANS_PER_HOUR=20 configurable via .env. Add slowapi for production enforcement.
CORS	Currently open (*) for local dev. Lock to specific origins in production.
Production Hardening	Add JWT authentication middleware, HTTPS via nginx SSL, secrets management (not .env), audit logging.

17. Advanced Features Roadmap

<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'FEATURE' 'frags': [ParaFrag(__tag __='b', bold=1, fontName='Helvetica-Bold', fontSize=8, greek=0, italic=0, link=[], rise=0, text='FEATURE', textColor=Color(.545098,.580392,.619608,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>	<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'EFFORT' 'frags': [ParaFrag(__tag __='b', bold=1, fontName='Helvetica-Bold', fontSize=8, greek=0, italic=0, link=[], rise=0, text='EFFORT', textColor=Color(.545098,.580392,.619608,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>	<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'IMPLEMENTATION' 'frags': [ParaFrag(__tag __='b', bold=1, fontName='Helvetica-Bold', fontSize=8, greek=0, italic=0, link=[], rise=0, text='IMPLEMENTATION', textColor=Color(.545098,.580392,.619608,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>
Parallel AI analysis	Low	asyncio.Semaphore(3) + asyncio.gather()
AI response caching	Low	Dict cache by violation rule ID
Scheduled scans	Medium	APScheduler with cron triggers
Jira integration	Medium	jira library — create issues per violation
PDF export	Medium	weasyprint HTML→PDF conversion
Historical trend charts	Medium	Chart.js in frontend, query scan history
Screenshot annotations	High	Pillow to draw bounding boxes on violations
JWT authentication	Medium	python-jose + OAuth2PasswordBearer
Multi-user + RBAC	High	User table, role middleware, tenant isolation
Kubernetes deploy	High	Deployment + Service + PVC for Ollama models
Slack notifications	Low	httpx POST to Slack webhook on scan complete
VS Code extension	High	TypeScript extension calling AccessAI API

18. Testing Guide

```
# Run full test suite
pytest tests/ -v --asyncio-mode=auto

# With coverage report
pip install pytest-cov
pytest tests/ --cov=backend --cov-report=html
open htmlcov/index.html

# Run specific test
pytest tests/test_main.py::test_url_validation -v

# Test scanner directly
python -c "
import asyncio
from backend.scanner.axe_scanner import AccessibilityScanner
async def test():
    s = AccessibilityScanner()
    r = await s.scan('https://example.com')
    print(f'Score: {r["score"]}, Violations: {len(r["violations"])}')
asyncio.run(test())
"
```

Test Coverage

- GET /health — 200 response
- POST /api/scan/start with invalid URL — 400 error
- POST /api/scan/start with missing URL — 422 validation error
- GET /api/scan/nonexistent/status — 404 error
- Scanner severity mapping (axe impact → AccessAI severity)
- Score calculation algorithm with weighted violations
- URL validation: valid URLs, blocked schemes, empty strings
- JSON parser: clean JSON, markdown fences, embedded JSON, garbage input

19. Troubleshooting

Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'PROBLEM' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=8, greek=0, italic=0, link=[], rise=0, text='PROBLEM', textColor=Color(.545098,.580392,.619608,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'CAUSE' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=8, greek=0, italic=0, link=[], rise=0, text='CAUSE', textColor=Color(.545098,.580392,.619608,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'SOLUTION' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=8, greek=0, italic=0, link=[], rise=0, text='SOLUTION', textColor=Color(.545098,.580392,.619608,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph
Ollama offline (red dot)	Ollama not started	Run: ollama serve
Model not found	Model not pulled	Run: ollama pull llama3
Scan failed immediately	Windows event loop conflict	Use python main.py (not uvicorn)
Browser launch fails	Playwright browsers missing	Run: python -m playwright install chromium
Scan fails on Linux	Missing system libs	playwright install-deps chromium
CORS error in browser	Backend not on port 8000	Check: curl http://localhost:8000/health
No AI analysis shown	Ollama timeout	Try phi3:mini — fastest model
Screenshot not loading	data/screenshots dir missing	mkdir data\screenshots
Score always 0	axe-core failed to load	Check network; download axe.min.js locally
History not showing	Database not initialized	Delete data/accessai.db and restart

20. Resume-Ready Project Description

Portfolio & Interview Reference

One-Line Description

Built a production-grade AI-powered WCAG 2.1 accessibility testing platform with real-time violation detection, local LLM analysis, and enterprise reporting — 38 files, zero cloud dependencies.

Full Description (LinkedIn / CV)

AccessAI — Enterprise AI Accessibility Testing Agent

Architected and built a production-grade automated accessibility testing platform using Python (FastAPI), Playwright browser automation, axe-core WCAG 2.1 engine, and locally-hosted Large Language Models via Ollama. The system performs real-time WCAG 2.1 scanning, classifies violations by severity (Critical/High/Medium/Low), and uses AI prompt engineering to generate human-readable explanations, business impact assessments, and code-level remediation guidance.

Features a professional enterprise dashboard with real-time progress steps, animated accessibility score ring, interactive violation explorer with detail modals, AI analysis cards, executive summary, page screenshot tab, and downloadable HTML/JSON reports. Scan history stored in SQLite for trend analysis. Deployed with Docker Compose; CI/CD via GitHub Actions with automated accessibility regression testing.

Tech Stack Keywords

Python · FastAPI · Playwright · axe-core · Ollama · llama3 · deepseek-r1 · qwen2.5 · mistral · SQLite ·aiosqlite · Pydantic v2 · Docker · docker-compose · Nginx · GitHub Actions · WCAG 2.1 · HTML5 · CSS3 · Vanilla JS · ThreadPoolExecutor · Prompt Engineering · REST API · Accessibility Testing · AI/LLM Integration

Key Talking Points for Interviews

- Solved a non-trivial Windows asyncio + Playwright subprocess conflict using ThreadPoolExecutor
- Implemented prompt engineering for structured JSON output from open-source LLMs
- Built robust JSON extraction that handles markdown fences, embedded JSON, and malformed LLM output
- Designed a polling architecture with real-time progress steps and in-memory status store
- Used Pydantic v2 for type-safe request/response validation across all API endpoints
- Implemented WCAG severity classification with weighted scoring algorithm
- Zero external API dependencies — fully air-gapped, privacy-preserving design
- Enterprise UI with score ring animation, responsive design, dark/light theme

■ AccessAI

Enterprise AI Accessibility Testing Agent

100% Local • Open Source • WCAG 2.1 AA

Powered by Ollama • FastAPI • Playwright • axe-core
v1.0.0 — May 2026